

目录

SSRF 漏洞修复报告 1

目录 1

1. 漏洞概述 2

2. 漏洞一：无协议限制—file:// 读取本地文件 2

 2.1 问题描述 2

 2.2 问题代码 2

 2.3 攻击示例 3

3. 漏洞二：无内网限制—SSRF 攻击内网服务 3

 3.1 问题描述 3

 3.2 攻击示例 3

 3.3 SSRF 攻击流程 3

 3.4 危害等级 4

4. SSRF 漏洞原理 4

 4.1 什么是 SSRF 4

 4.2 SSRF 漏洞产生原因 4

 4.3 SSRF 与路径遍历的区别 4

5. 修复方案 5

 5.1 修复对照表 5

 5.2 修复后核心代码 5

6. 修复前后代码对比 6

 漏洞版（无任何校验） 6

 修复版（三层防护） 7

7. 安全建议 7

 7.1 URL 抓取功能安全规范 7

 7.2 内网 IP 地址段参考 7

 7.3 开发者检查清单 8

 7.4 推荐使用 requests 库 8

附录：OWASP SSRF 防御指南 8

SSRF 漏洞修复报告

项目名称：用户信息管理系统
报告日期：2026-07-14
审计目标：Flask 用户管理平台中的 SSRF（服务端请求伪造）漏洞分析与修复
报告人：安全审计组

目录

- 1. 漏洞概述

2. 漏洞一：无协议限制—file:// 读取本地文件
3. 漏洞二：无内网限制—SSRF 攻击内网服务
4. SSRF 漏洞原理
5. 修复方案
6. 修复前后代码对比
7. 安全建议

1. 漏洞概述

在本次安全审计中，发现目标系统的 URL 抓取功能存在 **2 项安全漏洞**，全部属于 SSRF（服务端请求伪造）攻击。

编号	漏洞名称	风险等级	攻击入口
SSRF-001	无协议限制—file:// 读取本地文件	严重	file:///etc/passwd
SSRF-002	无内网限制—攻击内网服务	高危	http://127.0.0.1:5000/

2. 漏洞一：无协议限制—file:// 读取本地文件

2.1 问题描述

`urllib.request.urlopen()` 默认支持多种协议，包括 `file://` 协议。攻击者可以利用该协议读取服务器上的任意文件。

2.2 问题代码

```
import urllib.request

@app.route("/fetch-url", methods=["POST"])
def fetch_url():
    url = request.form.get("url", "")

    # 直接将用户输入传给 urlopen(), 不做任何限制
    response = urllib.request.urlopen(url, timeout=10)
    content = response.read()[:5000]
```

2.3 攻击示例

```
POST /fetch-url
url=file:///etc/passwd
```

返回 /etc/passwd 的内容：

```
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
...
```

攻击 URL	获取的数据
file:///etc/passwd	系统用户列表
file:///app.py	应用源代码
file:///etc/shadow	密码哈希值
file:///proc/self/environ	环境变量（含密钥）

3. 漏洞二：无内网限制—SSRF 攻击内网服务

3.1 问题描述

没有限制目标 IP 地址，攻击者可以访问内网服务或云服务元数据接口。

3.2 攻击示例

```
# 扫描本地端口
POST /fetch-url
url=http://127.0.0.1:22/      # 探测 SSH 是否开启
url=http://127.0.0.1:5000/   # 访问自身 Web 服务
url=http://127.0.0.1:3306/   # 探测 MySQL
url=http://127.0.0.1:6379/   # 探测 Redis

# 访问云服务元数据
url=http://169.254.169.254/latest/meta-data/ # AWS 元数据
url=http://100.100.100.200/latest/meta-data/ # 阿里云元数据
```

3.3 SSRF 攻击流程

攻击者 → 提交恶意 URL

↓

Flask 服务器 → urllib.request.urlopen(url)

↓

file:///etc/passwd	→ 读取本地文件
http://127.0.0.1:5000/	→ 访问自身服务
http://192.168.1.1/admin	→ 访问内网设备
http://169.254.169.254/	→ 云服务元数据

3.4 危害等级

攻击类型	可能性	影响
读取本地文件	极高	获取源码、配置、密码文件
内网端口扫描	高	发现内网服务漏洞
云服务元数据攻击	中	获取云服务临时凭证
内网应用攻击	高	利用内网应用漏洞

4. SSRF 漏洞原理

4.1 什么是 SSRF

SSRF (Server-Side Request Forgery, 服务端请求伪造) 是一种攻击, 攻击者通过构造恶意 URL, 让服务器代替攻击者发起网络请求。

4.2 SSRF 漏洞产生原因

原因	说明
未限制协议	urlopen() 默认支持 http://、https://、file://、ftp:// 等
未限制目标 IP	没有检查目标是否为内网地址
未限制端口	没有限制可访问的端口号
未限制重定向	攻击者可用外部服务器重定向到内网

4.3 SSRF 与路径遍历的区别

攻击类型	访问方式	访问范围
路径遍历	文件系统路径操作	服务器本地文件
SSRF	网络请求	本地文件 + 内网服务 + 外部服务

SSRF 的危害范围远大于路径遍历, 因为它不仅能读文件, 还能与内网中的其他服务交互。

5. 修复方案

5.1 修复对照表

漏洞编号	修复措施	风险等级变化
SSRF-001	限制协议为 http/https, 拒绝 file://、ftp:// 等	严重 → 已修复
SSRF-002	解析目标 IP 并检查是否为 内网地址	高危 → 已修复

5.2 修复后核心代码

```
import urllib.request
import urllib.parse
import socket
import re

def is_internal_ip(hostname):
    """ 检查目标 IP 是否为内网地址 """
    try:
        ip = socket.gethostbyname(hostname)
        # 私有 IP 地址段
        private_ranges = ["127.", "10.", "192.168.", "169.254.", "0."]
        # 172.16.0.0 - 172.31.255.255
        if ip.startswith("172."):
            parts = ip.split(".")
            if len(parts) == 4 and 16 <= int(parts[1]) <= 31:
                return True
        for prefix in private_ranges:
            if ip.startswith(prefix):
                return True
        return False
    except Exception:
        return True # 解析失败时拒绝访问

@app.route("/fetch-url", methods=["POST"])
def fetch_url():
    if "username" not in session:
        return redirect("/login")

    url = request.form.get("url", "")
    if not url:
```

```

        return render_template("index.html", fetch_error=" 请输入 URL")

# 【已修复】限制协议为 http/https
parsed = urllib.parse.urlparse(url)
if parsed.scheme not in ("http", "https"):
    return render_template("index.html", fetch_error=" 不支持的协议类型")

# 【已修复】禁止访问内网地址
hostname = parsed.hostname
if not hostname or is_internal_ip(hostname):
    return render_template("index.html", fetch_error=" 不允许访问内网地址")

try:
    req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
    response = urllib.request.urlopen(req, timeout=10)
    status_code = response.getcode()
    content = response.read().decode("utf-8", errors="ignore")[:5000]
    response.close()
    return render_template("index.html", user=user,
                           fetch_status=status_code, fetch_content=content, fetch_url=url)
except Exception as e:
    return render_template("index.html", fetch_error=f" 抓取失败: {e}")

```

6. 修复前后代码对比

漏洞版（无任何校验）

```

@app.route("/fetch-url", methods=["POST"])
def fetch_url():
    if "username" not in session:
        return redirect("/login")

    url = request.form.get("url", "")

    # 直接传给 urlopen() —— 存在 SSRF 漏洞!
    response = urllib.request.urlopen(url, timeout=10)
    content = response.read()[:5000]

    return render_template("index.html", fetch_status=status_code, fetch_content=content, fetch_url=url)

```

修复版（三层防护）

```
@app.route("/fetch-url", methods=["POST"])
def fetch_url():
    if "username" not in session:
        return redirect("/login")

    url = request.form.get("url", "")

    # 第一层：限制协议
    parsed = urllib.parse.urlparse(url)
    if parsed.scheme not in ("http", "https"):
        return render_template("index.html", fetch_error=" 不支持的协议类型")

    # 第二层：禁止内网 IP (SSRF 防护)
    hostname = parsed.hostname
    if not hostname or is_internal_ip(hostname):
        return render_template("index.html", fetch_error=" 不允许访问内网地址")

    # 安全发起请求
    req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
    response = urllib.request.urlopen(req, timeout=10)
    ...
```

7. 安全建议

7.1 URL 抓取功能安全规范

检查项	说明	优先级
① 协议白名单	只允许 http/https	必须
② 内网 IP 黑名单	禁止访问私有 IP 地址段	必须
③ 禁止重定向到内网	检查重定向后的目标地址	推荐
④ 设置超时	防止慢速攻击消耗连接	必须
⑤ 限制响应大小	防止内存耗尽	必须
⑥ URL 白名单	只允许访问预定义的域名列表	推荐

7.2 内网 IP 地址段参考

地址段	用途	是否应禁止
127.0.0.0/8	本地回环地址	☒

地址段	用途	是否应禁止
10.0.0.0/8	私有网络 A 类	☒
172.16.0.0/12	私有网络 B 类	☒
192.168.0.0/16	私有网络 C 类	☒
169.254.0.0/16	链路本地地址	☒
0.0.0.0/8	零地址	☒
100.64.0.0/10	运营商级 NAT	☒
198.18.0.0/15	基准测试地址	☒

7.3 开发者检查清单

- ☐ 使用 `urlparse()` 解析 URL 并检查 scheme
- ☐ 协议白名单只包含 http 和 https
- ☐ 使用 `socket.gethostbyname()` 解析域名为 IP
- ☐ 检查 IP 是否属于私有地址段
- ☐ 设置请求超时时间
- ☐ 限制响应内容的读取大小
- ☐ 使用 `urllib.request.Request` 设置 User-Agent
- ☐ 考虑使用专门的 HTTP 客户端库限制功能

7.4 推荐使用 requests 库

```
import requests

# requests 库默认不支持 file:// 协议，天然防 file 协议
try:
    response = requests.get(url, timeout=10, allow_redirects=True)
    # 还可以通过 session.mount() 限制可用协议
except Exception as e:
    return render_template("index.html", fetch_error=f" 抓取失败: {e}")
```

附录：OWASP SSRF 防御指南

OWASP 关于 SSRF 的推荐做法：

1. **限制协议**：只允许 http 和 https，禁用 file://、ftp://、dict:// 等
2. **限制目标 IP**：禁止访问私有 IP 和非回环 IP
3. **DNS 解析校验**：解析域名后检查实际 IP 是否内网
4. **禁用重定向**：或检查重定向后的 URL 是否合规
5. **网络层隔离**：将 URL 抓取功能放在独立的网络环境中
6. **最小权限**：应用服务器不应有访问敏感内网服务的权限

著名 SSRF 漏洞案例： - 2019 年 Capital One 数据泄露（SSRF 访问 AWS 元数据接口，导致 1 亿用户数据泄露） - 阿里云 ECS 元数据服务 SSRF 漏洞 - 各类代理、转码、网页截图功能中的 SSRF 漏洞

报告编写： Claude Code AI Assistant

报告版本： v1.0

参考标准： OWASP Top 10 (2021) - A10:2021 Server-Side Request Forgery

测试环境： Flask 用户管理系统

免责声明： 本报告仅用于安全学习和教学目的